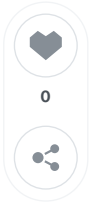


5. 합성곱 신경망 (2)

goblurry · 21분 전

통계 수정 삭제

딥러닝



딥러닝 파이토치 교과서

▼ 목록 보기

3/3



5.3 전이 학습
5.3.1 특성 추출
5.3.2 미세 조정
5.4 설명 가능한
5.4.1 특성 맵
5.5 그래프 합성
5.5.1-2 그래프
5.5.3 그래프

 딥러닝 파이토치 교과서 5장 '합성곱 신경망'의 5.3, 5.4절의 내용을 정리한 포스트입니다. 5장은 두 개의 포스트로 나누어 정리합니다. <https://velog.io/@hyunsun/5.-합성곱-신경망-1> 에서 이전 내용을 확인할 수 있습니다.

- 5.1 합성곱 신경망
- 5.2 합성곱 신경망 맛보기
- 5.3 전이 학습
- 5.4 설명 가능한 CNN
- 5.5 그래프 합성곱 네트워크

5.3 전이 학습

CNN 기반의 딥러닝 모델을 제대로 훈련시키기 위해서는 많은 데이터가 필요하다. 하지만 충분히 큰 데이터셋을 얻는 것은 비용상 어렵다. 이를 해결할 수 있는 방법으로 전이 학습이 있다.

전이 학습이란, 아주 큰 데이터셋(ex. ImageNet)을 이용해 훈련된 모델의 **가중치**를 가져와, 우리가 해결하고자 하는 문제에 맞게 보정하여 사용하는 방식을 의미한다. 이때 그 훈련된 모델을 **사전 훈련된 모델**이라고 한다. 비교적 적은 수의 데이터를 사용해 과제를 해결할 수 있다는 장점이 있다.

전이 학습을 위한 방법으로 **특성 추출**과 **미세 조정 기법**이 있다.

5.3.1 특성 추출 기법 Feature Extract

특성 추출 기법은 ImageNet 데이터셋으로 사전 훈련된 모델을 가져온 뒤, 마지막에 **완전연결층** 부분(이미지 카테고리 결정)만 새로 만든다. 즉, 학습 시에는 마지막 완전연결층만 학습하고, 나머지 계층들은 학습되지 않도록 한다.

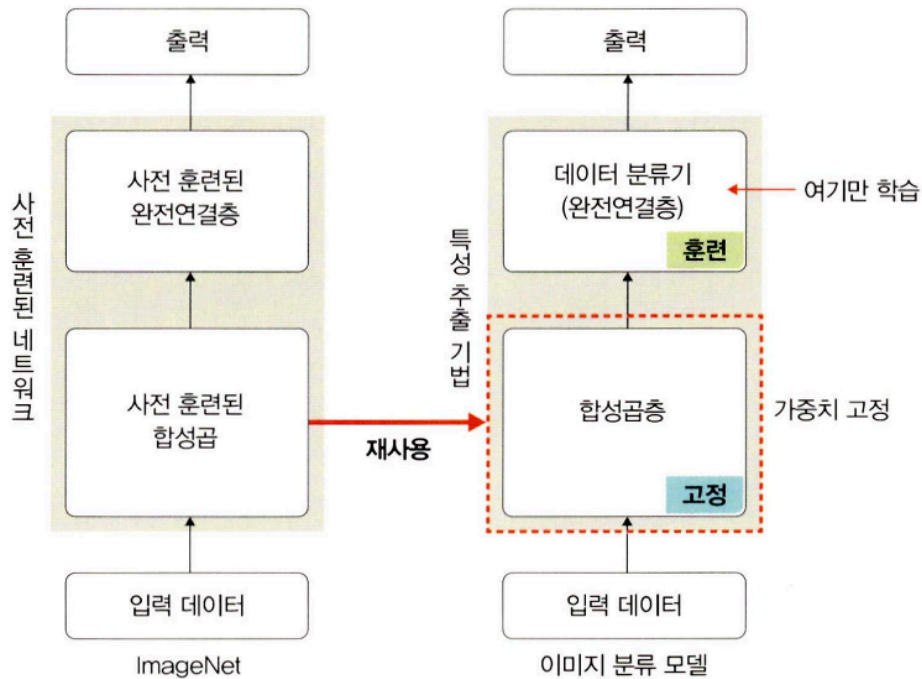
특성 추출은 이미지 분류를 위해 다음과 같이 두 부분으로 구성된다.

- 합성곱층: 합성곱층 + 풀링층
- 완전연결층(데이터 분류기): 추출된 특성을 인풋으로 받아, 이미지에 대한 클래스를 분류하는 부분

사전 훈련된 모델의 합성곱층에 새로운 데이터를 통과시키고, 그 출력값을 데이터 분류기(FC층)에서 훈련시킨다.

사용 가능한 이미지 분류 모델에는 Xception, Inception V3, ResNet50, VGG16, MobileNet 등이 있다.

▼ 그림 5-30 특성 추출 기법



실습

교재의 특성 추출 실습에 대해 간단히 설명하려고 한다. 고양이와 강아지 이미지를 이용해 클래스를 분류하는 문제이다. 전체 소스 코드는 https://github.com/goblurry/DL_study 에서 볼 수 있다.

우선 CNN 모델로는 **ResNet18**을 사용했다. 이는 50개 계층으로 이루어진 합성곱 신경망으로, ImageNet DB의 100만 개가 넘는 영상을 이용해 훈련된 신경망으로 전이 학습에 사용되도록 사전 훈련된 모델을 제공한다.

특히 이 실습에서는 합성곱 계층의 가중치를 고정하고, 마지막 완전연결층만 학습하도록 설정하였다.

이는 기존 모델이 학습한 특징을 그대로 추출하여 활용하고, 새로운 분류 문제에 맞게 출력층만 조정하는 방식이다.

1. 모델 다운로드

```
# download pre-trained model
resnet18 = models.resnet18(pretrained=True)

# ResNet18의 학습곱층을 사용하되 파라미터에 대해서는 학습하지 않도록 고정
def set_parameter_requires_grad(model, feature_extracting=True):
    if feature_extracting:
        for param in model.parameters():
            # 역전파 중 파라미터에 대한 변화 계산이 불필요함을 나타냄
            # 모델의 합성곱층과 풀링층을 고정하고 나머지를 학습
            param.set_parameter_requires_grad = False;

set_parameter_requires_grad(resnet18)
```

모델의 일부(합성곱층, 풀링층)를 고정하고 나머지를 학습하고자 할 때 **requires_grad=False**

2. 모델의 마지막 부분에 완전연결층 추가: 개와 고양이 클래스 분류하는 용도

```
resnet18.fc = nn.Linear(512, 2) # 2: 클래스가 두 개라는 의미
```

3. 모델 생성

```
# 모델 객체 생성
model = models.resnet18(pretrained=True)

for param in model.parameters():
    param.requires_grad = False # 학습 x: 합성곱층 가중치 고정

model.fc = torch.nn.Linear(512, 2)
for param in model.fc.parameters():
    param.requires_grad = True # 완전연결층은 학습

# 손실 함수 정의
optimizer = torch.optim.Adam(model.fc.parameters())
cost = torch.nn.CrossEntropyLoss()
```

이렇게 정의하면 기존의 합성곱 계층은 고정된 상태로 유지되고, 마지막 완전연결층만 학습 대상이 된다.

4. 모델 학습

학습 과정에서는 입력 이미지를 모델에 통과시켜 예측값을 얻고, 실제 레이블과의 차이를 손실 함수로 계산한 뒤 역전파를 통해 가중치를 업데이트한다. 이때 업데이트되는 부분은 완전연결층에 한정된다.

```
for epoch in range(num_epochs):
    for inputs, labels in dataloader:
        outputs = model(inputs)
        loss = cost(outputs, labels)
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()
```

이 과정이 반복되면서, 기존에 학습된 특징 위에 새로운 분류 기준이 점진적으로 형성된다.

5. 학습 결과

```

Epoch 5/12
-----
Loss: 0.2375 Acc: 0.8987

Epoch 6/12
-----
Loss: 0.2252 Acc: 0.9195

Epoch 7/12
-----
Loss: 0.2160 Acc: 0.9273

Epoch 8/12
-----
Loss: 0.1934 Acc: 0.9247

Epoch 9/12
-----
Loss: 0.1755 Acc: 0.9506

Epoch 10/12
-----
Loss: 0.2239 Acc: 0.9143

Epoch 11/12
-----
Loss: 0.1899 Acc: 0.9195

Epoch 12/12
-----
Loss: 0.1683 Acc: 0.9455

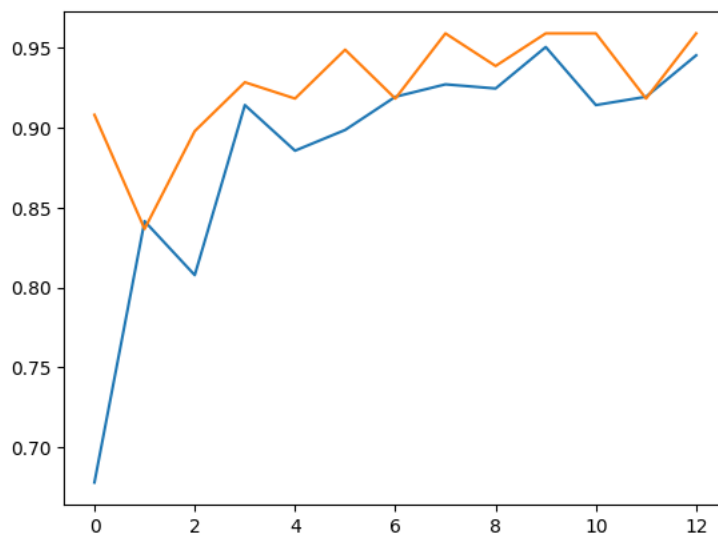
Training complete in 6m 27s
Best Acc: 0.950649

```

학습 과정에서 에포크가 증가함에 따라 정확도가 점진적으로 향상되는 것을 확인할 수 있다. 이는 사전 학습된 특징을 기반으로 새로운 데이터에 맞는 분류 경계가 점차 잘 형성되고 있음을 의미한다.

또한 전체 모델을 재학습하지 않고 일부 계층만 학습하였음에도 성능이 개선되는 점에서, 사전 학습된 특징이 다양한 이미지 분류 문제에 효과적으로 활용될 수 있음을 확인할 수 있다.

6. 모델 평가



파랑: 훈련 데이터에 대한 정확도 / 주황: 검증 데이터에 대한 정확도

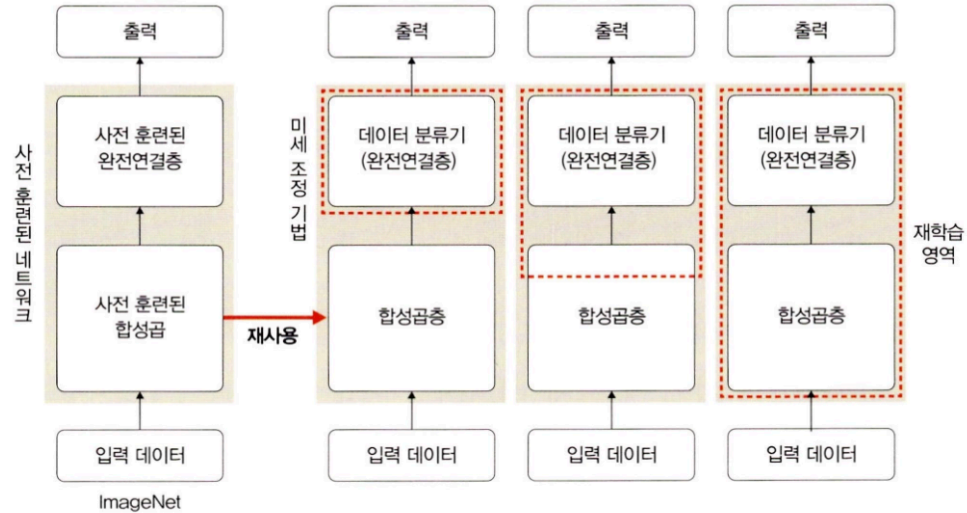
학습이 완료된 이후에는 테스트 데이터를 이용하여 모델의 성능을 평가하였다. 학습에 사용되지 않은 데이터에서도 높은 정확도를 보이는 것을 통해, 모델이 단순히 학습 데이터에 과적합된 것이 아니라 일반적인 패턴을 잘 학습했음을 확인할 수 있다. 또한, 두 데이터 모두 에포크가 진행될수록 정확도가 향상되고 있다. 더 많은 에포크 진행 시에 100%에 수렴할 것임을 알 수 있다.

5.3.2 미세 조정 기법 Fine Tuning

미세 조정 기법(이하 파인튜닝)은 특성 추출 기법에서 나아가, **사전 훈련된 모델과 합성곱층, 데이터 분류기의 가중치**를 업데이트하여 훈련시키는 방식이다.

즉, '사전 학습된 모델'을 미세 조정하여 분석하고자 하는 데이터셋에 맞게 파라미터를 조정하는 기법이다.
(연산량이 많기 때문에 CPU보다 GPU를 사용할 것이 권장된다.) 이때 파라미터에 큰 변화를 줄 경우 과적합이 발생할 수 있으므로 미세한 업데이트가 필요하다.

♥ 그림 5-44 미세 조정 기법



훈련하고자 하는 데이터셋의 크기와 사전 훈련된 모델에 따라 다음과 같은 전략을 세울 수 있다.

데이터셋 크기	사전 학습 모델과의 유사성	학습 전략	이유
큼	작음	모델 전체 재학습	기존 특징이 도움이 되지 않으므로 전체를 새로 학습해야 함
큼	큼	후반부 계층(FC 인접) + 분류기 학습	상위 계층에서 task-specific 특징만 조정하면 충분
작음	작음	일부 합성곱층 + 분류기 학습	데이터는 적지만 특징 차이가 커 일부는 재학습 필요
작음	큼	분류기만 학습	데이터 부족 → 전체 학습 시 과적합 위험

세 번째 케이스의 경우, 데이터가 적어서 일부분에 파인튜닝을 적용해도 효과가 없을 수 있다. 합성곱층 중 어떤 부분을 새롭게 학습시킬지를 적당히 설정해야 한다.

5.4 설명 가능한 CNN

딥러닝 처리 결과를 사람이 이해할 수 있는 방식으로 제시하는 기술을 **설명 가능한 CNN**이라고 한다.

CNN은 내부 동작 과정을 알기가 어려워서 그 결과 역시 신뢰하기 어렵다. 이 문제를 해결하기 위해 CNN을 구성하는 각각의 중간 계층부터 최종 분류까지, 이미지 인풋에서 특성이 어떻게 추출되고 학습하는지를 시각화하는 것이다.

CNN 시각화 방법으로는 '필터에 대한 시각화'와 '특성 맵에 대한 시각화'가 있는데, 이 교재에서는 특성 맵에 대한 시각화만 다룬다.

5.4.1 특성 맵 시각화

특성 맵(=활성화 맵): 필터를 적용한 결과 이미지 (입력 이미지에서 특정한 특징만 강조된 결과)

CNN의 경우, 입력 이미지에 필터를 적용해 얻은 특성 맵에 또 필터를 적용해서 그에 대한 특성 맵을 연쇄적으로 얻을 수 있다. 특성 맵을 시각화함으로써 특성 맵에서 입력 특성을 감지하는 방법을 이해할 수 있다.

교재의 특성 맵 시각화 실습에 대해 간단히 설명하려고 한다. 전체 소스 코드는

https://github.com/goblurry/DL_study 에서 볼 수 있다.

설명 가능한 모델을 위해 **13개의 합성곱층과 2개의 완전연결층**으로 구성된 네트워크를 생성하였으며, 두 계층은 ReLU 활성화 함수를 사용한다.

- XAI 모델 정의: 합성곱 계층을 반복적으로 쌓아 입력 이미지로부터 점진적으로 고차원 특징을 추출한다. 각 블록은 Conv-BatchNorm-ReLU 구조를 따르며, 일부 구간에는 드롭아웃을 적용하여 과적합을 방지하였다. 또한 MaxPooling을 통해 특성 맵의 크기를 줄이면서 중요한 특징만 유지하도록 설계하였다. 최종적으로 추출된 특성은 완전연결층을 통해 분류 결과로 변환된다.
- *Forward 함수: 입력 이미지는 합성곱 기반의 특성 추출기를 통과하며 다양한 수준의 특징을 추출한다. 이 과정에서 낮은 수준의 정보부터 점차 복잡한 패턴까지 계층적으로 학습된다. 이후 특성 맵은 1차원 벡터로 변환(flatten)되며, 완전연결층을 통해 각 클래스에 대한 점수로 변환된다. 마지막으로 출력 함수(로그 소프트맥스)를 통해 각 클래스에 대한 확률 값이 계산된다.

본 실험에서는 아래의 고양이 입력 이미지를 모델에 통과시켜, 중간 합성곱 계층에서 생성되는 특성 맵을 시각화하였다. 이를 통해 모델이 이미지의 어떤 특징을 학습하고 있는지를 확인하고자 하였다.



구체적으로는 모델 내부에서 **특징이 어떻게 변화하는지**를 확인하기 위해, 특정 합성곱 계층의 출력을 직접 추출하였다. 특성 추출기의 레이어 인덱스를 기준으로 0번째, 20번째, 40번째를 선택하고, 해당 지점의 특성 맵을 시각화하였다.

1. 0번째 Conv2d의 특성 맵

```
# (0): Conv2d(3, 64, kernel_size=(3, 3), stride=(1, 1), bias=False)
result = LayerActivations(model.features, 0)

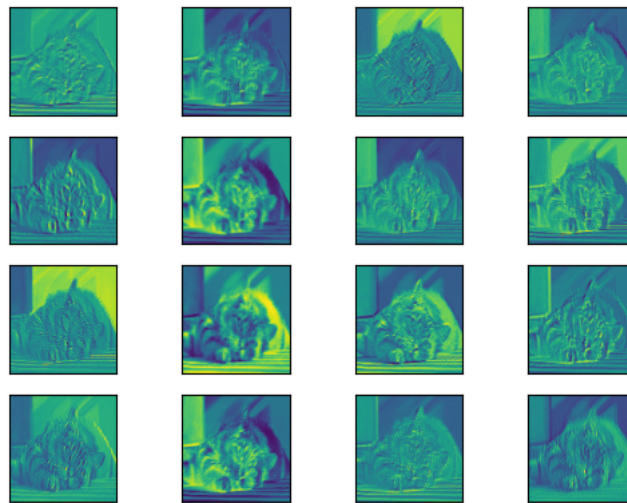
model(img)
activations = result.features
```

첫 번째 라인의 0 위치에 원하는 인덱스를 입력하면 된다.

```
# 특성 맵 시각적으로 확인
fig, axes = plt.subplots(4,4)
fig = plt.figure(figsize=(12, 8))
fig.subplots_adjust(left=0, right=1, bottom=0, top=1, hspace=0.05, wspace=0.05)

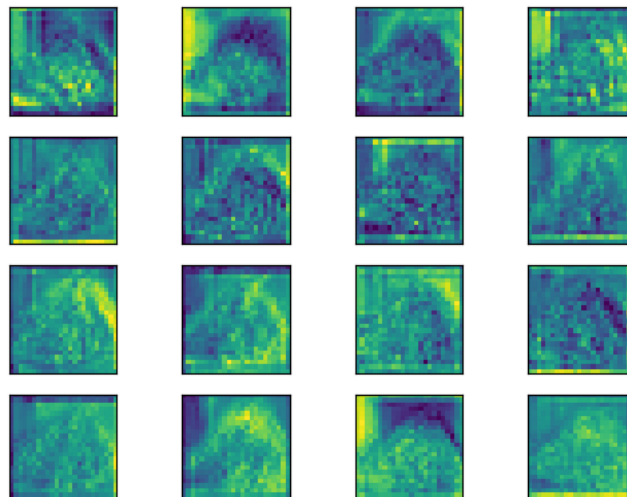
for row in range(4):
    for column in range(4):
        axis = axes[row][column]
        axis.get_xaxis().set_ticks([])
        axis.get_yaxis().set_ticks([])
        axis.imshow(activations[0][row*10+column])
plt.show()
```

[시각화 결과]



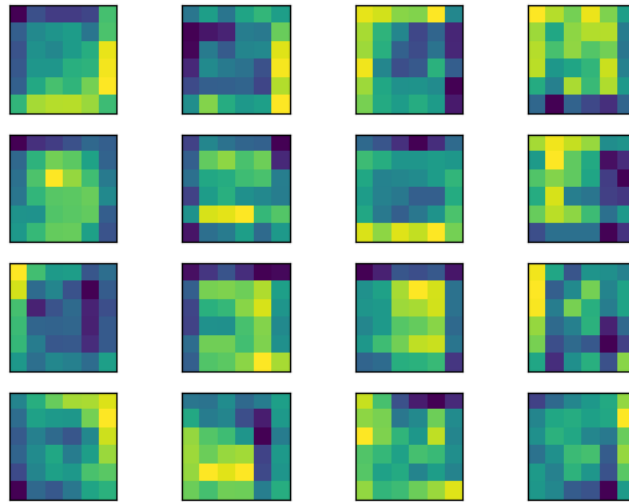
0번째 Conv2d의 경우 입력층과 인접하기 때문에 비교적 원본 입력 이미지의 형태를 유지하고 있음을 확인할 수 있다.

2. 20번째 Conv2d 특성 맵



기존 고양이 이미지의 형태는 잘 드러나지 않는다.

3. 40번째 Conv2d 특성 맵



40번째 맵에서는 입력 이미지의 형태를 찾아볼 수 없게 되었다. 출력층에 가까울수록 기존 형태는 찾아볼 수 없고 이미지 특징들만 전달되는 것을 확인할 수 있다.

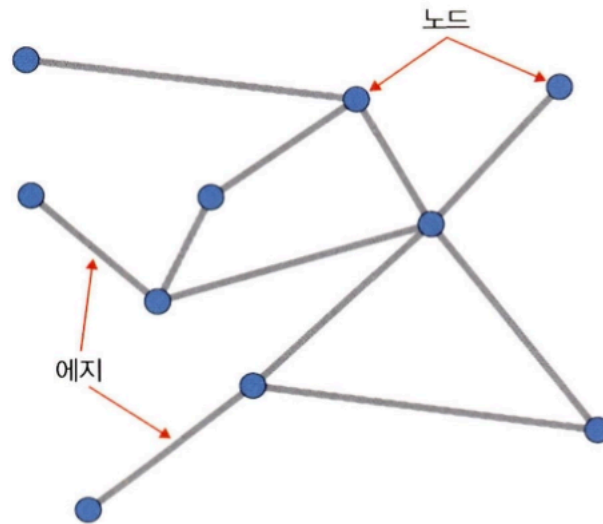
☀️ 깊은 계층으로 갈수록 특성 맵에서 원본 이미지의 형태가 점점 희미해지는 이유는, 모델이 저수준의 시각적 정보가 아닌 고차원적이고 추상적인 특징을 중심으로 표현을 재구성하기 때문이다. 즉, 단순한 픽셀 정보가 아니라 객체를 구성하는 의미 있는 패턴들이 강조되면서, 시각적으로는 원본과 다른 형태로 나타나게 된다.

(채널 수 증가는 표현의 다양성을 의미하며, 추상성의 증가는 계층의 깊이에 의해 결정된다.)

5.5 그래프 합성곱 네트워크

5.5.1-2 그래프와 그래프 신경망

▼ 그림 5-50 그래프



그래프란 위 이미지와 같은 형태를 취하는 것으로, edge(연결선)로 연결된 node(=vertex, 원소)들의 집합인데, 이 에지는 방향성을 가질 수도 가지지 않을 수도 있다.

이러한 그래프 구조에서 사용하는 신경망을 **그래프 신경망, Graph Neural Network**라고 한다. 그래프 데이터에 대한 표현은 **[인접 행렬 → 특성 행렬]** 두 단계로 이루어진다.

▼ 그림 5-51 특성 행렬⁵



1. 인접 행렬

노드 n 개를 가지는 네트워크를 $n \times n$ 행렬로 표현한 뒤, 행렬 내의 값 A_{ij} 를 i 와 j 의 관련성 여부를 만족하는 값으로 채워 준다.

예를 들어 위 그림에서 네트워크는 노드 5개를 가지므로 5×5 의 행렬을 생성한다. 노드 1은 노드 2, 노드 3과 연결되어 있으므로 (1, 2), (1, 3) (2, 1), (3, 1)에 1을 채워넣고 연결되어 있지 않은 4와 5에는 0을 채워넣는 것이다.

이 과정은 컴퓨터가 이해하기 쉽게 그래프로 표현하는 과정이다.

2. 특성 행렬

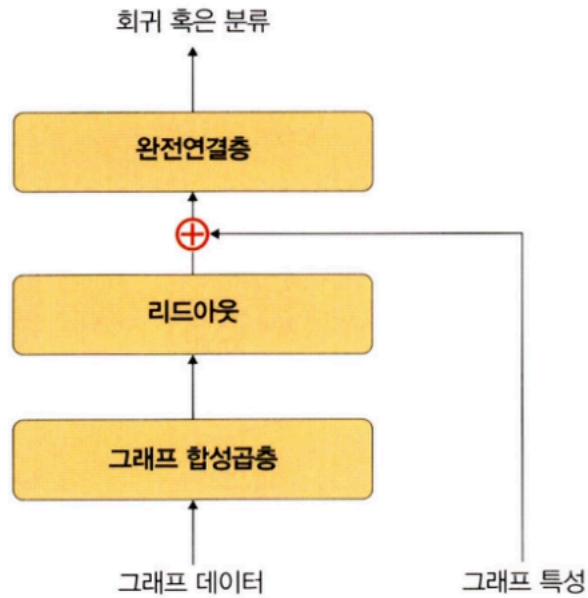
인접 행렬만으로는 특성을 충분하게 파악하기 어렵기 때문에 단위 행렬을 적용한다. 각 입력 데이터에서 이용할 특성을 선택하는데, 특성 행렬에서 각 행은 선택된 특성에 대해 각 노드가 가지는 값을 의미한다.

이 두 과정을 거쳐 **그래프의 특성**이 추출된다.

5.5.3 그래프 합성곱 네트워크 GCN

그래프 합성곱 네트워크는 이미지에 대한 합성곱을 그래프 데이터로 확장한 알고리즘이다. 구조는 다음과 같다.

▼ 그림 5-52 그래프 합성곱 네트워크



리드아웃(readout): 특성 행렬을 하나의 벡터로 변환하는 함수. 전체 노드의 특성 벡터에 대해 평균을 구하고 그래프 전체를 표현하는 하나의 벡터를 생성한다.

GCN에서 가장 중요한 부분은 **그래프 합성곱층**이다. 기존의 그래프 형태의 데이터로는 딥러닝 알고리즘을 적용할 수 없는데, 그래프 합성곱층을 이용한 그래프 형태의 데이터는 **행렬** 형태의 데이터로 변환되기 때문에 딥러닝 알고리즘을 적용할 수 있다.

GCN은 SNS의 관계 네트워크, 학술 연구의 인용 네트워크, 3D Mesh(모델링, 프린팅, 가상현실 등 구현하는 핵심 기술) 등의 분야에서 활용된다.



goblurry



이전 포스트

5. 합성곱 신경망 (1)

0개의 댓글

댓글을 작성하세요

댓글 작성

